BLOCKSEC

Security Audit
Report for EIP7702 Contracts

Date: July 28, 2026 **Version:** 1.0
Contact: contact@blocksec.com

Contents

Report Manifest

Item	Description
Client	AllScale
Target	EIP7702 Contracts

Version History

Version	Date	Description
1.0	July 28, 2026	First release

Signature

Digitally signed by: BlockSec (https://blocksec.com)

SHA1 digest of public key: BAB106D3708BC6DE0963CE05E9478C24B326AE1C

Date: 2026-08-05 03:00:05+0000

About BlockSec BlockSec focuses on the security of the blockchain ecosystem and collaborates with leading DeFi projects to secure their products. BlockSec is founded by top-notch security researchers and experienced experts from both academia and industry. They have published multiple blockchain security papers in prestigious conferences, reported several zero-day attacks of DeFi applications, and successfully protected digital assets that are worth more than 14 million dollars by blocking multiple attacks. They can be reached at Email, Twitter and Medium.

Chapter 1 Introduction

1.1 About the Audit Target

Information	Description
Type	Smart Contract
Language	Solidity
Approach	Semi-automatic and manual verification

The audit target (hereinafter referred to as the Target) of this audit is provided as a **ZIP** archive.

EIP7702 Contracts of AllScale is a gasless wallet infrastructure built on EIP7702 account delegation. It lets whitelisted sponsors relay signed EIP712 authorizations on behalf of EOAs, executing fee-split token transfers and deposits. All wallet logic runs through a three-layer beacon proxy pattern, enabling seamless logic upgrades without changing the delegated contract. Specifically, EOAs delegate to the contract BeaconStub, which resolves the active implementation (i.e., the contract EIP7702) from an upgradeable beacon contract. Then, the contract BeaconStub delegatecalls into the implementation to execute transfers.

Note this audit only focuses on the smart contracts in the following directories/files:

- BeaconStub.sol
- EIP7702.sol
- GlobalConfig.sol
- foundry/src/BeaconStub.sol

Other files are not within the scope of the audit. Additionally, all dependencies of the Target are considered reliable in terms of both functionality and security, and are therefore not included in the audit scope.

The auditing process is iterative. Specifically, we would audit the commits that fix the discovered issues. If there are new issues, we will continue this process. The MD5 SHA values during the audit are shown in the following table. Our audit report is responsible for the code in the initial version (`Version 1`), as well as new code (in the following versions) to fix issues in the audit report. Code prior to and including the baseline version (`Version 0`), where applicable, is outside the scope of this audit and assumes to be reliable and secure.

Project	Version	MD5 SHA
eip7702	`Version 1`	`37193ced9fe435c87b200edb103e6599`
	`Version 2`	`a3d4c59f1dc1b5aea7f634375297c90e`

1.2 Disclaimer

This audit report does not constitute investment advice or a personal recommendation. It does not consider, and should not be interpreted as considering or having any bearing on, the potential economics of a token, token sale or any other product, service or other asset.

Any entity should not rely on this report in any way, including for the purpose of making any decisions to buy or sell any token, product, service or other asset.

This audit report is not an endorsement of any particular project or team, and the report does not guarantee the security of any particular project. This audit does not give any warranties on discovering all security issues of the Target, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit cannot be considered comprehensive, we always recommend proceeding with independent audits and a public bug bounty program to ensure the security of the Target.

The scope of this audit is limited to the code mentioned in Section 1.1. Unless explicitly specified, the security of the language itself (e.g., the solidity language), the underlying compiling toolchain and the computing infrastructure are out of the scope.

1.3 Procedure of Auditing

We perform the audit according to the following procedure.

- **Vulnerability Detection** We first scan the Target with automatic code analyzers, and then manually verify (reject or confirm) the issues reported by them.
- **Semantic Analysis** We study the business logic of the Target and conduct further investigation on the possible vulnerabilities using an automatic fuzzing tool (developed by our research team). We also manually analyze possible attack scenarios with independent auditors to cross-check the result.
- **Recommendation** We provide some useful advice to developers from the perspective of good programming practice, including gas optimization, code style, and etc.

We show the main concrete checkpoints in the following.

1.3.1 Security Issues

* Access control
* Permission management
* Whitelist and blacklist mechanisms
* Initialization consistency
* Improper use of the proxy system
* Reentrancy
* Denial of Service (DoS)
* Untrusted external call and control flow
* Exception handling
* Data handling and flow
* Events operation
* Error-prone randomness
* Oracle security
* Business logic correctness
* Semantic and functional consistency
* Emergency mechanism

* Economic and incentive impact

1.3.2 Additional Recommendation

* Gas optimization
* Code quality and style

Note *The previous checkpoints are the main ones. We may use more checkpoints during the auditing process according to the functionality of the project.*

1.4 Security Model

To evaluate the risk, we follow the standards or suggestions that are widely adopted by both industry and academy, including OWASP Risk Rating Methodology [1] and Common Weakness Enumeration [2]. The overall *severity* of the risk is determined by *likelihood* and *impact*. Specifically, likelihood is used to estimate how likely a particular vulnerability can be uncovered and exploited by an attacker, while impact is used to measure the consequences of a successful exploit.

In this report, both likelihood and impact are categorized into two ratings, i.e., *high* and *low* respectively, and their combinations are shown in Table 1.1.

Table 1.1: Vulnerability Severity Classification

Impact \ Likelihood	*High*	*Low*
High	High	Medium
Low	Medium	Low

Accordingly, the severity measured in this report are classified into three categories: **High**, **Medium**, **Low**. For the sake of completeness, **Undetermined** is also used to cover circumstances when the risk cannot be well determined.

Furthermore, the status of a discovered item will fall into one of the following five categories:

- **Undetermined** No response yet.
- **Acknowledged** The item has been received by the client, but not confirmed yet.
- **Confirmed** The item has been recognized by the client, but not fixed yet.
- **Partially Fixed** The item has been confirmed and partially fixed by the client.
- **Fixed** The item has been confirmed and fixed by the client.

[1] https://owasp.org/www-community/OWASP_Risk_Rating_Methodology

[2] https://cwe.mitre.org/

Chapter 2 Findings

In total, we found **one** potential security issue. Besides, we have **two** recommendations and **one** note.

- High Risk: 1
- Recommendation: 2
- Note: 1

ID	Severity	Description	Category	Status
1	High	Potential fee circumvention due to malicious delegations	Security Issue	Fixed
2	-	Remove the redundant function `receive()`	Recommendation	Confirmed
3	-	Add checks for the value `s` in the function `_recover()`	Recommendation	Fixed
4	-	Potential centralization risks	Note	-

The details are provided in the following sections.

2.1 Security Issue

2.1.1 Potential fee circumvention due to malicious delegations

Severity High

Status Fixed in `Version 2`

Introduced by `Version 1`

Description In the protocol, sponsors are allowed to pay gas fees for users' transfers via the adoption of EIP7702. In the contract `EIP7702`, the `gaslessTransfer()` and `gaslessApproveAndDeposit()` functions transfer a portion of users' tokens to existing fee recipients as fees. However, users can front-run sponsors' gasless transactions by delegating to malicious contracts (via EIP7702). As a result, users can circumvent fees of gasless transactions with a malicious delegation.

```
313   // ===== Business entry (global-config driven) =====
314   function gaslessTransfer(
315       address token,
316       address to,
317       uint256 amount,
318       uint256 feeAmount,
319       uint256 deadline,
320       bytes32 id,
321       bytes calldata signature
322   ) external nonReentrant nonPaused {
323       require(amount > 0, "ZeroAmount");
324       require(token != address(0) && to != address(0), "ZeroAddr");
325       require(address(token).code.length > 0, "TokenNotContract");
326       require(feeAmount <= amount, "FeeGTAmount");
327       _requireDeadline(deadline);
328
```

```
329         // Sponsor allowlist
330         require(GLOBAL_CONFIG.isSponsorAllowed(msg.sender), "SPONSOR_DENY");
331
332         // Optional token allowlist
333         if (GLOBAL_CONFIG.useTokenAllowlist()) {
334             require(GLOBAL_CONFIG.isTokenAllowed(token), "TOKEN_DENY");
335         }
336
337         // id anti-replay
338         AccountStorage storage s = _getAccountStorage();
339         require(!s.usedId[id], "ID_USED");
340         s.usedId[id] = true;
341
342         // Verify signature & bump opNonce. opNonce no longer participates in
343         // the EIP-712 digest -anti-replay is provided by ``id`` + ``usedId``;
344         // the storage counter is kept purely for event indexing / debugging.
345         (bytes32 digest, uint256 usedNonce) = _verifyAndBumpOp(
346             keccak256(
347                 abi.encode(
348                     TRANSFER_TYPEHASH,
349                     msg.sender,
350                     token,
351                     to,
352                     amount,
353                     feeAmount,
354                     deadline,
355                     id
356                 )
357             ),
358             signature
359         );
360
361         // Transfers
362         IERC20Like t = IERC20Like(token);
363         address feeRecvForEvent = GLOBAL_CONFIG.feeRecipient();
364         require(feeRecvForEvent != address(0), "FEE_ZERO");
365
366         if (feeAmount > 0) {
367             if (GLOBAL_CONFIG.enforceExactNet()) {
368                 _transferExactNet(t, feeRecvForEvent, feeAmount);
369             } else {
370                 t.safeTransfer(feeRecvForEvent, feeAmount);
371             }
372         }
```

Listing 2.1: eip7702/EIP7702.sol

```
386     function gaslessApproveAndDeposit(
387         address token,
388         address depositContract,
389         uint256 amount,
390         uint256 feeAmount,
391         uint256 commitmentId,
```

```
392         uint256 deadline,
393         bytes32 id,
394         bytes calldata signature
395     ) external nonReentrant nonPaused {
396         // ---- Param validation (same shape as gaslessTransfer) ----
397         require(amount > 0, "ZeroAmount");
398         require(token != address(0) && depositContract != address(0), "ZeroAddr");
399         require(address(token).code.length > 0, "TokenNotContract");
400         require(address(depositContract).code.length > 0, "DepositNotContract");
401         // ``amount`` here is the buyer's total spend; the contract splits it
402         // into ``feeAmount`` (skimmed to the AllScale fee wallet) and
403         // ``amount - feeAmount`` (forwarded to Rhino's depositWithId, which
404         // must equal the quote-time payAmount). ``feeAmount > amount`` would
405         // make the deposit leg underflow →reject early.
406         require(feeAmount <= amount, "FeeGTAmount");
407         _requireDeadline(deadline);
408 
409         // ---- Sponsor allowlist ----
410         require(GLOBAL_CONFIG.isSponsorAllowed(msg.sender), "SPONSOR_DENY");
411 
412         // ---- Optional token allowlist ----
413         if (GLOBAL_CONFIG.useTokenAllowlist()) {
414             require(GLOBAL_CONFIG.isTokenAllowed(token), "TOKEN_DENY");
415         }
416 
417         // ---- id anti-replay ----
418         AccountStorage storage s = _getAccountStorage();
419         require(!s.usedId[id], "ID_USED");
420         s.usedId[id] = true;
421 
422         // ---- Verify EIP-712 signature, bump opNonce ----
423         // opNonce no longer participates in the digest (anti-replay = ``id``);
424         // _verifyAndBumpOp still increments the storage counter for events.
425         (bytes32 digest, uint256 usedNonce) = _verifyAndBumpOp(
426             keccak256(
427                 abi.encode(
428                     APPROVE_DEPOSIT_TYPEHASH,
429                     msg.sender,
430                     token,
431                     depositContract,
432                     amount,
433                     feeAmount,
434                     commitmentId,
435                     deadline,
436                     id
437                 )
438             ),
439             signature
440         );
441 
442         // ---- AllScale Pay wallet-business fee (mirrors gaslessTransfer) ----
443         // Skim BEFORE the deposit leg so a deposit revert (e.g. Rhino route
444         // outage) atomically rolls back the fee transfer too -never leaves a
```

```
445       // "fee taken but bridge not started" inconsistency.
446       address feeRecv = address(0);
447       if (feeAmount > 0) {
448           feeRecv = GLOBAL_CONFIG.feeRecipient();
449           require(feeRecv != address(0), "FEE_ZERO");
450           if (GLOBAL_CONFIG.enforceExactNet()) {
451               _transferExactNet(IERC20Like(token), feeRecv, feeAmount);
452           } else {
453               IERC20Like(token).safeTransfer(feeRecv, feeAmount);
454           }
455       }
```

Listing 2.2: eip7702/EIP7702.sol

Impact Users can front-run gasless transactions by malicious delegations to circumvent fees.

Suggestion Revise the logic accordingly.

Clarification from BlockSec The project introduced the contract `SponsorGateway` to check the EOA's 7702 delegations in gasless transactions. The contract `SponsorGateway` will be whitelisted in the contract `GlobalConfig`, and only the role `OPERATOR` (i.e., the actual sponsor) will be allowed to access the contract `SponsorGateway` to send gasless transactions. Moreover, the project ensures proper configurations (i.e., policy, operator, and user status) by introducing the roles `GOVERNANCE` (i.e., the project's Safe wallet) and `STATUS_MANAGER` (i.e., an independent role).

2.2 Recommendation

2.2.1 Remove the redundant function `receive()`

Status Confirmed

Introduced by `Version 1`

Description In the contract `GasSponsoredAccount7702`, the function `receive()` is redundant. It is recommended to remove it.

```
152   receive() external payable {}
```

Listing 2.3: eip7702/EIP7702.sol

Suggestion Remove the redundant function `receive()`.

Feedback from the project The project confirmed the finding and stated that the redundant function `receive()` in the contract `GasSponsoredAccount7702` will be removed in the future.

2.2.2 Add checks for the value `s` in the function `_recover()`

Status Fixed in `Version 2`

Introduced by `Version 1`

Description In the contract `GasSponsoredAccount7702`, the functions `_tryRecover()` and `_recover()` implement similar signature recovery logic. However, the function `_tryRecover()` additionally validates the value `s` with the check `uint256(s) <= _SECP256K1_HALF_N`, while the function

`_recover()` does not perform this check. It is recommended to add checks for the value `s` in the function `_recover()`.

```
219   function _tryRecover(bytes32 digest, bytes calldata signature) internal pure returns (address
          signer) {
220       bytes32 r;
221       bytes32 s;
222       uint8 v;
223
224       if (signature.length == 65) {
225           assembly ("memory-safe") {
226               r := calldataload(signature.offset)
227               s := calldataload(add(signature.offset, 32))
228               v := byte(0, calldataload(add(signature.offset, 64)))
229           }
230       } else if (signature.length == 64) {
231           bytes32 vs;
232           assembly ("memory-safe") {
233               r := calldataload(signature.offset)
234               vs := calldataload(add(signature.offset, 32))
235           }
236           v = uint8((uint256(vs) >> 255) + 27);
237           s = bytes32(uint256(vs) & ((1 << 255) - 1));
238       } else {
239           return address(0);
240       }
241
242       if (uint256(s) > _SECP256K1_HALF_N || (v != 27 && v != 28)) {
243           return address(0);
244       }
```

Listing 2.4: eip7702/EIP7702.sol

```
253   function _recover(bytes32 digest, bytes memory signature) internal pure returns (address) {
254       bytes32 r;
255       bytes32 s;
256       uint8 v;
257       if (signature.length == 65) {
258           assembly ("memory-safe") {
259               r := mload(add(signature, 32))
260               s := mload(add(signature, 64))
261               v := byte(0, mload(add(signature, 96)))
262           }
263       } else if (signature.length == 64) {
264           assembly ("memory-safe") {
265               r := mload(add(signature, 32))
266               s := mload(add(signature, 64))
267           }
268           v = uint8((uint256(s) >> 255) + 27);
269           s = bytes32(uint256(s) & ((1 << 255) - 1));
270       } else {
271           revert("BAD_SIG_LEN");
272       }
273       require(v == 27 || v == 28, "BAD_V");
```

```
274         address signer = ecrecover(digest, v, r, s);
275         require(signer != address(0), "ECRECOVER_ZERO");
276         return signer;
277     }
```

Listing 2.5: eip7702/EIP7702.sol

Suggestion Add checks for the value `s` in the function `_recover()`.

2.3 Note

2.3.1 Potential centralization risks

Introduced by `Version 1`

Description In this project, several privileged roles (e.g., `owner`) can conduct sensitive operations, which introduces potential centralization risks. For example, the owner of the contract `GlobalConfig` can update critical configuration through several functions. If the private keys of the privileged accounts are lost or maliciously exploited, it could pose a significant risk to the protocol.

Feedback from the project The project stated that the operational configuration and incident-response permissions are managed through restricted KMS credentials. Additionally, beacon upgrades are controlled by a 2-of-3 Safe multisig.


BOOST WEB3 THROUGH NEXT-GENERATION SECURITY & USABILITY INNOVATIONS